

TRAINING SESSION

DevOps & Azure DevOps

From Culture to Pipeline — A Complete Introduction

Inspired by Gene Kim · Donovan Brown · The DevOps Handbook

1

Today's Agenda

DevOps Introduction

- What is DevOps?
- Culture & Practices
- CI, CD & Pipelines

Azure DevOps Deep Dive

- Repos, Pipelines
- Build & Release
- Multi-stage Pipelines

Live Walkthrough

- Azure DevOps portal
- Creating pipelines
- End-to-end flow

Hands-On Lab

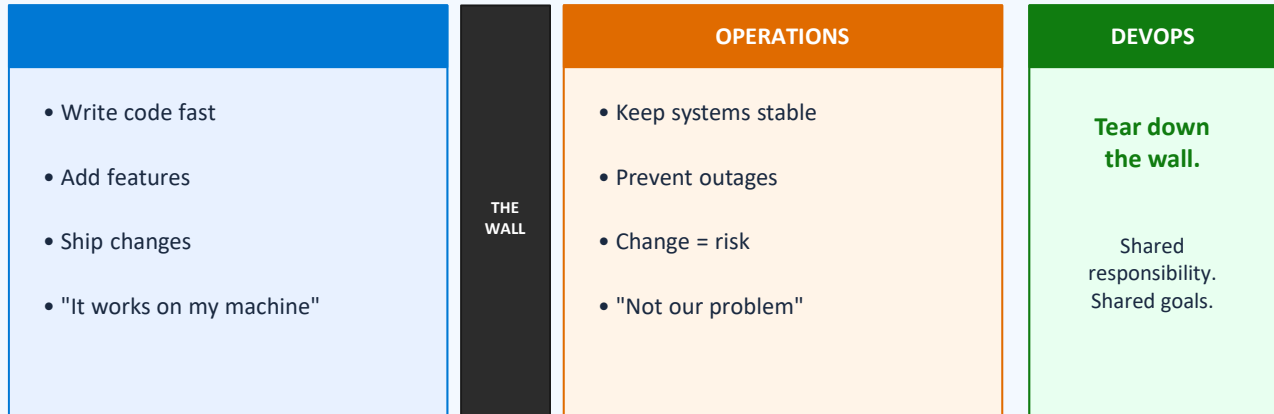
- Your own pipeline
- Guided exercises
- Q&A

1

2

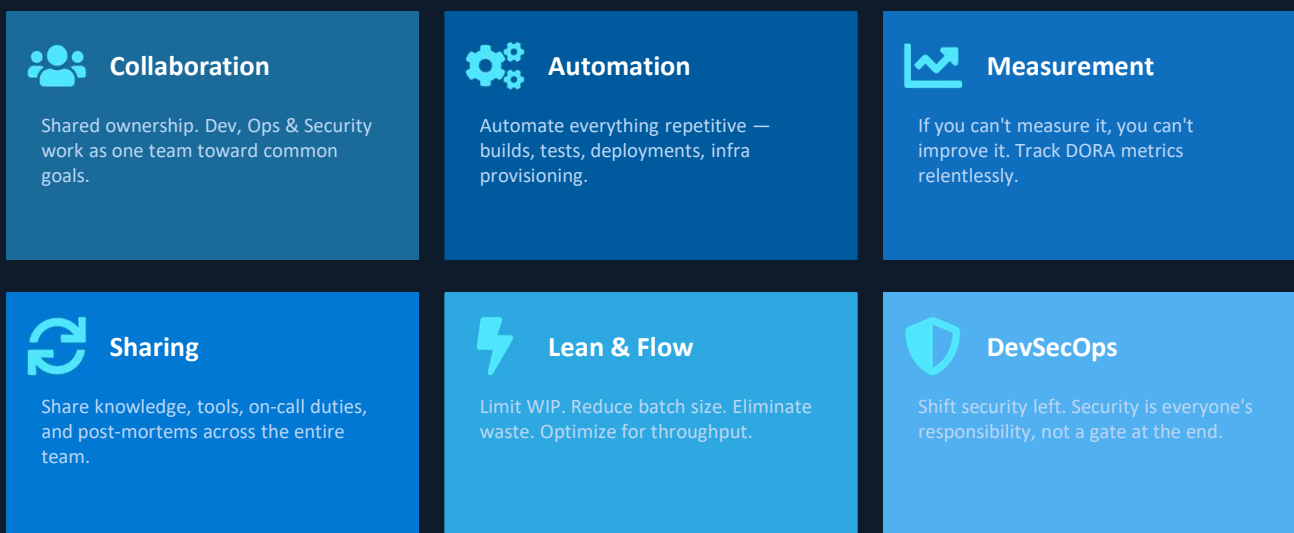
Section 1

What is DevOps?



3

DevOps Culture & Practices

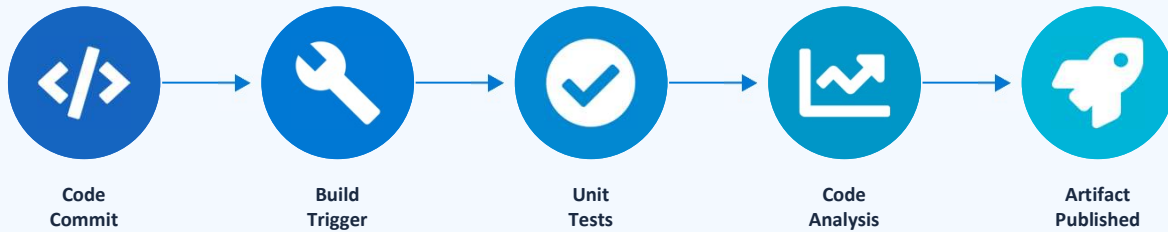


2

4

Continuous Integration (CI)

"Integrate early, integrate often. The longer you wait, the worse the merge conflict."



Trunk-Based Dev

Commit to main/trunk frequently.
Feature branches kept short-lived (< 1 day).

Fast Feedback

CI build must complete in < 10 minutes.
Developers wait for green before moving on.

Fix the Build First

Broken build = team emergency. Nobody commits on top of a broken build.

5

Continuous Delivery (CD)

Continuous DELIVERY

Software is ALWAYS in a deployable state.
Deployment to production is a BUSINESS decision, not a technical one.
Requires human approval to deploy.

Continuous DEPLOYMENT

Every change that passes all tests is deployed to production AUTOMATICALLY.
No human gates. Requires exceptional test coverage.
Used by Netflix, Amazon, Etsy.

Deployment Strategies

Blue/Green

Two identical envs, instant switch

Canary

Gradually route traffic to new version

Rolling

Replace instances incrementally

Feature Flags

Toggle features without deployment

3

6

CI/CD Pipeline — End to End



7

SECTION 2

Azure DevOps

**One platform.
Every stage of your
pipeline.**

Azure DevOps is Microsoft's cloud-based platform providing integrated tools for planning, coding, building, testing, and deploying software — for ANY language, ANY platform, ANY cloud.

40M+

developers

Free

up to 5 users

Any

cloud / on-prem

Azure Boards

Agile planning & tracking

Azure Repos

Git version control

Azure Pipelines

CI/CD automation

Azure Test Plans

Manual & automated testing

Azure Artifacts

Package management

4

8

Azure Repos

Git-powered version control for your team

Unlimited Private Repos

Host unlimited Git repositories. Free for teams up to 5 users.

Branch Policies

Require PR reviews, linked work items, passing builds before merging to main.

Pull Request Workflows

Code review with inline comments, suggested changes, and vote policies.

Protected Branches

Lock down main/production branches. No direct pushes — everything goes through PRs.

Git Flow at a Glance

- **main**
Production-ready, always
- **develop**
Integration branch
- **feature/***
Short-lived, < 1 day
- **release/***
Stabilization only
- **hotfix/***
Emergency fixes to main

9

Azure Pipelines

Build, test, and deploy — automatically. To any platform. Any language.

YAML Pipelines ✓ Recommended

- ✓ Pipeline-as-code (lives in your repo)
- ✓ Version controlled with your code
- ✓ Reusable templates across projects
- ✓ PR-reviewable pipeline changes
- ✓ Full multi-stage support

Classic Pipelines (Legacy)

- GUI-based editor
- Good for beginners
- No code required
- Limited to single-stage by default
- Harder to track changes over time

Works with everything:

.NET	Node.js	Java	Python	Go	Ruby
PHP	Docker	K8s	Azure	AWS	GCP

5

10

Build Pipelines

Compile · Test · Package · Publish

```
azure-pipelines.yml

trigger:
- main

pool:
  vmImage: ubuntu-latest

variables:
  buildConfig: Release

steps:
- task: DotNetCoreCLI@2
  displayName: Restore
  inputs:
    command: restore
- task: DotNetCoreCLI@2
  displayName: Build
  inputs:
    command: build
- task: PublishBuildArtifacts@1
```

Triggers

Push to branch, PR, scheduled, or manual. Control WHEN your pipeline runs.

Agent Pools

Microsoft-hosted (fresh VM per run) or Self-hosted (your own machine).

Tasks

1000+ pre-built tasks in the marketplace. Restore, Build, Test, Publish, Docker, Terraform...

Variables & Secrets

Pipeline variables, variable groups, and Azure Key Vault integration for secrets.

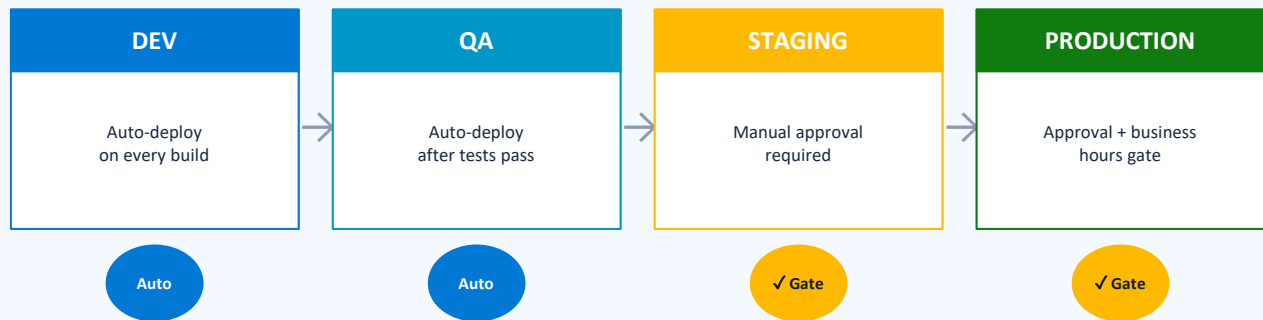
Artifacts

Build outputs published to Azure Artifacts or Pipeline artifact storage for deployment stages.

11

Release Pipelines

Structured deployment across environments with approvals and gates



Release Gates — Automated Quality Checks

Work Item Query

All linked bugs resolved before promoting?

Invoke REST API

External system health check passes?

Azure Monitor Alert

No active alerts in the target environment?

Business Hours

Only deploy between 9am–5pm on weekdays?

6

12

Multi-Stage Pipelines

Build + Release unified in ONE YAML file. The modern approach.

azure-pipelines.yml (multi-stage)

```
stages:
- stage: Build
  jobs:
  - job: BuildJob
    steps:
    - script: dotnet build
    - task: PublishArtifact

- stage: DeployDev
  dependsOn: Build
  jobs:
  - deployment: DeployDev
    environment: dev

- stage: DeployProd
  dependsOn: DeployDev
```

Single Source of Truth

Build and deploy live in one file, version-controlled alongside app code.

Environments & Approvals

Approval gates, exclusive locks, and deployment history built into YAML environments.

Stage Dependencies

dependsOn + condition controls flow. Parallel stages supported for faster pipelines.

Templates & Reuse

Extract common jobs into templates. Share across 50 repositories with one update.



Pro Tip: Use Environments to configure approvers, branch policies, and deployment history visualization in the Azure DevOps portal.

13

Measuring DevOps Success

The DORA Four Key Metrics — backed by 7 years of research across 33,000+ organizations



7

14

What's Next



30-Minute Live Demo

1. Create an Azure DevOps project from scratch
2. Set up Azure Repos and clone locally
3. Write a multi-stage YAML pipeline
4. Configure branch policies and approvals
5. Watch the pipeline run end to end



30-Minute Hands-On Lab

1. Fork the lab starter repo (link provided)
2. Create your own build pipeline
3. Add a deploy stage to a staging environment
4. Configure an approval gate
5. Trigger a deployment and approve it

15

Key Takeaways

Remember these five.

01

DevOps is Culture First

Tools amplify your culture. Fix the culture before buying the platform.

02

Automate the Repetitive

Every manual step is a risk. Automate builds, tests, and deployments.

03

Pipeline-as-Code

Your YAML pipeline is code. Version it, review it, test it.

04

Gates Protect Production

Approvals, quality gates, and branch policies are your safety net.

05

Measure with DORA

Speed and stability aren't in conflict. High performers have both.

dev.azure.com · aka.ms/learndevops · *The Phoenix Project* · *Accelerate*

8

16